

Шпаргалка по Python

Основы синтаксиса, коллекции и срезы, функции, классы, файлы и исключения, окружения и полезные модули стандартной библиотеки. Примеры для Python 3.9 и новее.

Типы и переменные

```
x = 42; y = 3.14; s = "текст"; flag = True; n = None
int, float, str, bool, NoneType — объявлять тип заранее не нужно

type(x), isinstance(x, int)
Узнать тип; проверить принадлежность типу

int("42"), float("3.14"), str(42), bool("")
Преобразования; пустая строка, 0 и пустая коллекция — ложь

a, b = b, a
Обмен значений без временной переменной

x: int = 5
Аннотация типа: подсказка для редактора и линтера, на выполнение не влияет
```

Строки

```
f"Привет, {name}! Итого: {total:.2f}"
f-строка: подстановка выражений и форматирование

s.strip(), s.lower(), s.title()
Убрать пробелы по краям, регистр

s.split(", "), ", ".join(items)
Разбить на список и собрать обратно

s.replace("a", "b"), s.startswith("http")
Замена подстроки, проверка начала (есть и endswith)

"подстрока" in s
Поиск вхождения; индекс даст s.find() или s.index()

"""многострочный текст"""
Строка в несколько строк; r"\d+" — без обработки обратных слешей
```

Коллекции

```
nums = [3, 1, 2]
Список: изменяемый, упорядоченный

point = (10, 20)
Кортеж: неизменяемый, годится как ключ словаря

uniq = {1, 2, 3}
Множество: только уникальные значения, быстрые проверки вхождения

user = {"name": "Аня", "age": 30}
Словарь: пары ключ — значение

nums.append(4), nums.extend([5, 6]), nums.pop()
Добавить элемент, добавить несколько, снять последний

sorted(nums, key=len, reverse=True)
Новый отсортированный список; nums.sort() сортирует на месте

user.get("email", "нет")
Значение по ключу с запасным вариантом вместо ошибки

for k, v in user.items():
Обход словаря; keys() и values() — только ключи или значения
```

Срезы

```
s[0], s[-1]
Первый и последний элемент

s[1:4]
С первого по третий: правая граница не включается

s[:3], s[3:]
Начало и хвост последовательности

s[::2], s[::-1]
Каждый второй элемент; развернуть
```

Ветвления и циклы

```
if score > 90:
    grade = "отлично"
elif score > 70:
    grade = "хорошо"
else:
    grade = "нужно повторить"

for i, item in enumerate(items, start=1):
    print(i, item)

for name, score in zip(names, scores):
    ...

while queue:
    task = queue.pop()
```

```
break / continue
Прервать цикл; перейти к следующей итерации

range(0, 10, 2)
Последовательность чисел с шагом

match value: case ...:
Сопоставление с образцом, доступно с версии 3.10
```

Генераторы коллекций

```
[x * 2 for x in nums if x > 0]
Список из преобразованных и отфильтрованных элементов

{w: len(w) for w in words}
Словарь: ключ — слово, значение — длина

{x % 3 for x in nums}
Множество уникальных значений

sum(x * x for x in nums)
Генераторное выражение: считает по мере обхода, не занимая память
```

Функции

```
def greet(name: str, greeting: str = "Привет") -> str:
    """Возвращает приветствие."""
    return f"{greeting}, {name}!"

def total(*args, **kwargs):
    ...
```

***args / **kwargs**

Произвольное число позиционных и именованных аргументов

lambda x: x * 2

Короткая безымянная функция — обычно как аргумент key=

yield

Превращает функцию в генератор: значения отдаются по одному

Ловушка: изменяемое значение по умолчанию (def f(items=[])) создаётся один раз и сохраняется между вызовами. Пишите items=None и создавайте список внутри.

Классы

```
class User:
    def __init__(self, name: str):
        self.name = name

    def __repr__(self) -> str:
        return f"User({self.name!r})"
```

```
from dataclasses import dataclass
```

```
@dataclass
class Point:
    x: int
    y: int = 0
```

@dataclass

Сам создаёт __init__, __repr__ и сравнение — меньше однотипного кода

@property

Метод, к которому обращаются как к атрибуту

@staticmethod / @classmethod

Без доступа к экземпляру; с классом первым аргументом

Файлы и контекст

```
from pathlib import Path

text = Path("data.txt").read_text(encoding="utf-8")

with open("out.csv", "w", encoding="utf-8") as f:
    f.write("a;b\n")

import json
data = json.loads(raw)
raw = json.dumps(data, ensure_ascii=False, indent=2)
```

Конструкция with закрывает файл даже при ошибке. Для путей удобнее pathlib: Path("dir") / "file.txt", .exists(), .glob("*.log").

Исключения

```
try:
    value = int(raw)
except ValueError as exc:
    print("не число:", exc)
except (KeyError, IndexError):
    ...
else:
    print("успех")
finally:
    cleanup()
```

raise ValueError("сообщение")

Возбудить исключение

raise RuntimeError("не вышло") from exc

Сохранить исходную причину в цепочке

Окружение и пакеты

python -m venv .venv

Создать виртуальное окружение в каталоге проекта

source .venv/bin/activate

Активировать (в Windows: .venv\Scripts\activate)

pip install -r requirements.txt

Установить зависимости из файла

pip freeze > requirements.txt

Зафиксировать текущие версии

python -m module

Запустить модуль как программу (http.server, json.tool, pytest)

Из стандартной библиотеки

- **pathlib** — пути и файлы. **os**, **sys**, **shutil** — система и оболочка.
- **json**, **csv**, **sqlite3** — данные и хранение.
- **datetime**, **zoneinfo** — даты и часовые пояса.
- **re** — регулярные выражения. **argparse** — разбор аргументов командной строки.
- **collections** (Counter, defaultdict, deque), **itertools**, **functools** (cache, partial).
- **logging** вместо print в рабочем коде; **unittest** и внешний **pytest** для тестов.